<u>README</u>

<u>Compiling the project:</u>

1. <u>Windows 11:</u>

1.1. <u>Option 1:</u>

To compile C code with GCC on Windows 11, install MinGW-w64, which includes the GCC compiler.

1. Download and install **MSYS2** from: msys2.org
2. Open the MSYS2 app/terminal and run:
   a. pacman -S mingw-w64-ucrt-x86_64-gcc
3. Add GCC to your system PATH: C:\msys64\ucrt64\bin
   a. In Windows Search, type "system variables" and select "Edit the system environment variables."
   b. Click "Environment Variables…". In the System variables section, scroll down and select "Path."
   c. Make sure you edit the existing Path variable, creating a new one will not work. Windows must update the existing PATH to find the command.
   d. Click "Edit" → "New", then add: "C:\msys64\ucrt64\bin".
   e. Click "OK" on all windows to save and close.
4. Then verify the installation by opening a new Command Prompt and running:
   a. gcc --version

To compile the project, run the following command inside the ./LOP/LOP directory, where all the C source files referenced in the command are located:

```
gcc instance.c optimization.c timer.c utilities.c algorithms.c cw.c vnd.c \
main_teacher.c main_lop.c main_vnd.c main.c -o LOP
```

After the build is complete, the executable (LOP.exe) will be located in the same ./LOP/LOP directory where you ran the gcc command.

1.2. <u>Option 2:</u>

To compile the project on Windows 11, you need to install Visual Studio.

1. Download and install **Visual Studio** from: https://visualstudio.microsoft.com/downloads/
2. Open the LOP.slnx project file in Visual Studio. (You can find it in the root LOP folder.)
3. In the top menu, click "Build", then select "Rebuild Solution" from the dropdown.

After the build is complete, the executable (LOP.exe) will be generated in the ./LOP/x64/Debug/ directory.

2. macOS

To compile C code on macOS, using GCC (Apple Clang):

1. Open a Terminal and install the Xcode Command Line Tools by running:
   a. xcode-select --install
2. Follow the popup instructions to complete the installation.
3. Then verify the installation by opening a new Terminal and running:
   a. gcc --version
   b. You should see output like: “Apple clang version ...”.

To compile the project, run the following command inside the ./LOP/LOP directory (On macOS, the top-level folder may be LOP-main depending on how the project was extracted.), where all the C source files referenced in the command are located:

```
gcc instance.c optimization.c timer.c utilities.c algorithms.c cw.c vnd.c \
main_teacher.c main_lop.c main_vnd.c main.c -o LOP
```

After the build completes, the executable (LOP, a Unix executable file) will be located in the same ./LOP/LOP (or ./LOP-main/LOP) directory where you ran the gcc command.

3. Linux

To compile C code on Linux, you need to install GCC (GNU Compiler Collection).

1. Install GCC.
   Depending on your distribution, use one of the following commands:
   a. **Ubuntu / Debian:**

      ```
      sudo apt update
      sudo apt install build-essential
      ```

   b. **Fedora:**

      ```
      sudo dnf install gcc
      ```

   c. **Arch Linux:**

      ```
      sudo pacman -S base-devel
      ```

2. Then
   verify the installation by opening a new Terminal and running:
   a. gcc --version

To compile the project, run the following command inside the ./LOP/LOP directory, where all the C source files referenced in the command are located:

```
gcc instance.c optimization.c timer.c utilities.c algorithms.c cw.c vnd.c \
main_teacher.c main_lop.c main_vnd.c main.c -o LOP
```

After the build completes, the executable (LOP) will be located in the same ./LOP/LOP directory where you ran the gcc command.

Running the Project:

1. Run a Single Instance

To run the project, use the following command format:

```
./LOP(.exe) -m <lop|vnd> -i <input_file> [-b <best_known_value>]
```

Notes:

- m <lop|vnd>: Selects which algorithm to run
  - lop: Runs iterative improvement using all 12 combinations of starting solution (Random, CW), neighbourhood (Transpose, Exchange, Insert), and pivot rule (First-improvement, Best-improvement).
  - vnd: Runs Variable Neighbourhood Descent using the CW starting solution, comparing two neighbourhood orderings (Transpose→Exchange→Insert and Transpose→Insert→Exchange).
- Replace <input_file> with the path to your instance file.
- Replace <value> with the most optimal known cost for that instance.
  - This is optional but allows the program to calculate deviation.
  - The optimal costs can be found in ./LOP/data/best_known/best_known.txt.
- The command must be executed from the directory containing LOP.exe (see above).

Open a command prompt, navigate to the directory containing LOP.exe, and run the command.

Example (Windows - Command Prompt):

```
C:\LOP\x64\Debug> ./LOP.exe -m lop-i ..\..\data\instances\N-be75eec_150 -b
```

Linux/Mac:

```
\LOP\LOP> ./LOP -m lop-i ..\data\instances\N-be75eec_150 -b 3482828
```

Warning:

If you compile the project using Visual Studio (as mentioned earlier), the generated .exe file will be located in ./LOP/x64/Debug/. This differs from compiling with gcc (on any operating system), where the executable is placed in ./LOP/LOP/.

Because of this difference, some of the file paths used in the Windows examples below may need to be adjusted when using gcc. For example, instead of:

..\..\data\instances (Visual Studio)

you should use:

..\data\instances (gcc)

Also note that you cannot blindly copy the commands as-is. You still need to replace the placeholder -m <lop|vnd> with the appropriate option, depending on whether you want to use lop or vnd.

2. Run All Instance Files

To run the program for all instance files in the instances folder:

Example (Windows - PowerShell):

```
Get-ChildItem "..\..\data\instances" | ForEach-Object {
  ./LOP.exe -m <lop|vnd> -i $_.FullName
}
```

Linux/Mac:

```
for f in ../data/instances/*; do ./LOP -m <lop|vnd> -i "$f"; done
```

3. Run All Instances with Best Known Values

To run all instance files in the instances folder with their associated optimal costs:

Example (Windows - PowerShell):

```
$best = @{}
 Get-Content "..\..\data\best_known\best_known.txt" | ForEach-Object {
   $parts = $_ -split "\s+"
   if ($parts.Count -ge 2) { $best[$parts[0]] = $parts[1] }
 }
 Get-ChildItem "..\..\data\instances" -File | ForEach-Object {
   $b = $best[$_.Name]
   if ($b) { ./LOP.exe -m <lop|vnd> -i $_.FullName -b $b }
   else    { ./LOP.exe -m <lop|vnd> -i $_.FullName }
 }
```

Linux/Mac:

```
for f in ../data/instances/*; do
  name=$(basename "$f")
  b=$(awk -v n="$name" '$1 == n {print $2}' ../data/best_known/best_known.txt)
  if [ -n "$b" ]; then
     ./LOP -m <lop|vnd> -i "$f" -b "$b"
  else
     ./LOP -m <lop|vnd> -i "$f"
    fi
  done
```

Project Structure Overview:

In addition to the executable file located in **./LOP/x64/Debug/**, the project includes two important result directories:

1. implementation_1_results:

This folder contains all files related to **Exercise 1**, which focuses on **basic iterative improvement search algorithms**.

Contents:

- **Instance result files (.csv)**
  For each instance problem provided in ./LOPdata/instances, a CSV file is generated.
  These files follow the naming convention: <instance-name>-result.csv
  For example: N-tiw56r72_250-result.csv
  Each file contains 12 results, corresponding to all combinations of:
  - Pivot rules: first improvement, best improvement
  - Neighborhoods: insert, exchange, transpose
  - Initial configurations: uninformed random picking, Chenery and Watanabe (CW)

- **combine.py**
  A Python script that reads all CSV result files in the current working directory and combines them into a single text file.

- **combined-results.txt**
  The aggregated results produced by running combine.py.

- **analysis_1.txt**
  Contains the output of statistical tests performed in R. This includes the average percentage deviation from the best known solution, the total execution time across all instances per algorithm setup, pairwise Student's t-test p-values (paired, two-sided) comparing all algorithm combinations, and Wilcoxon signed-rank test results identifying pairs with statistically significant performance differences ($p < 0.05$) along with which algorithm performed better in each case.

- **deviation_boxplot_1.pdf**
  A visualization generated in R, showing boxplots of the deviation percentage for each algorithm setup.

2. implementation_2_results:

This folder contains all files related to **Exercise 2**, which focuses on the **Variable Neighborhood Descent (VND)** algorithm.

Contents:

- **Instance result files (.csv)**
  For each instance problem in ./LOPdata/instances, a CSV file is generated.
  These files follow the same naming convention: <instance-name>-result.csv
  Each file contains 2 results, corresponding to the two possible orderings of neighborhood structures:
  - transpose → exchange → insert
  - transpose → insert → exchange

- **combine.py**
  A Python script that reads all CSV result files in the current working directory and combines them into a single text file.

- **combined-results.txt**
  The aggregated results produced by running combine.py.combined-results.txt with the combined results procuced by the python combine.py file

- **analysis_2.txt**
  Contains the output of statistical tests performed in R. This includes the average percentage deviation from the best known solution, the total execution time across all instances per algorithm setup, pairwise Student's t-test p-values (paired, two-sided) comparing all algorithm combinations, and Wilcoxon signed-rank test results identifying pairs with statistically significant performance differences ($p < 0.05$) along with which algorithm performed better in each case.

- **deviation_boxplot_2.pdf**
  A visualization generated in R, showing boxplots of the deviation percentage for each algorithm setup.

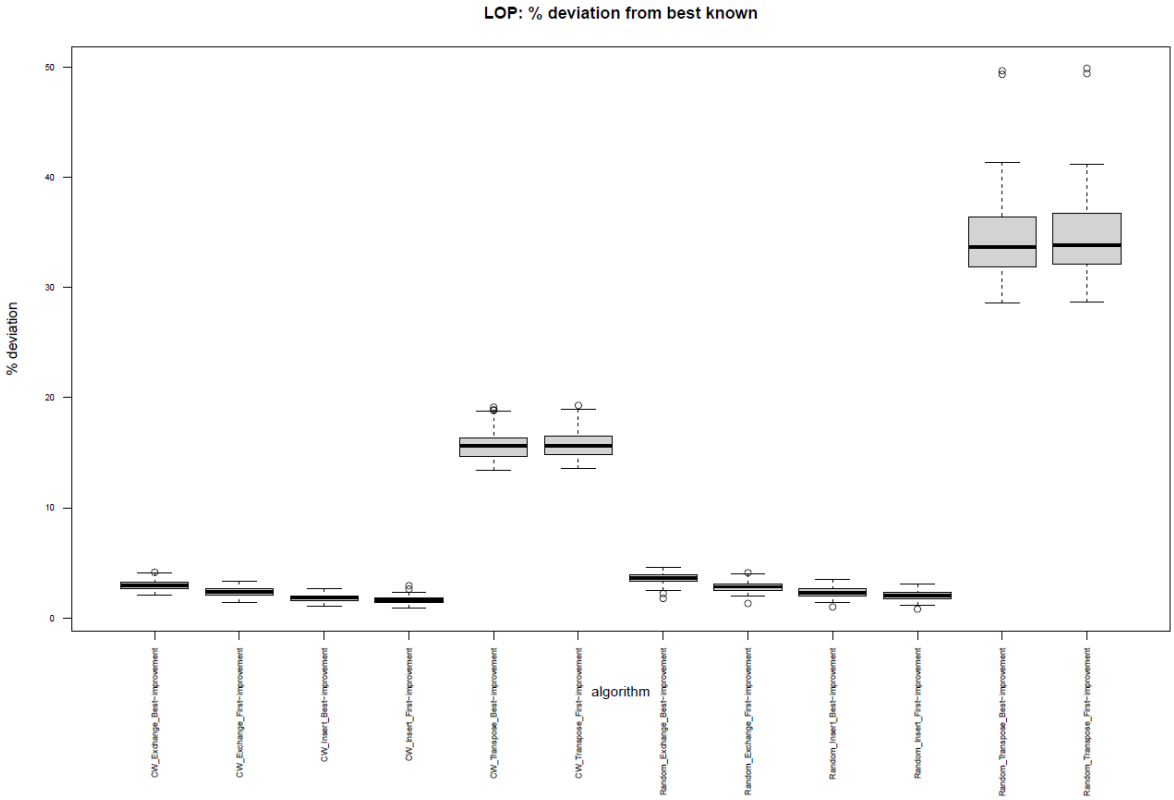
Source Code Location:

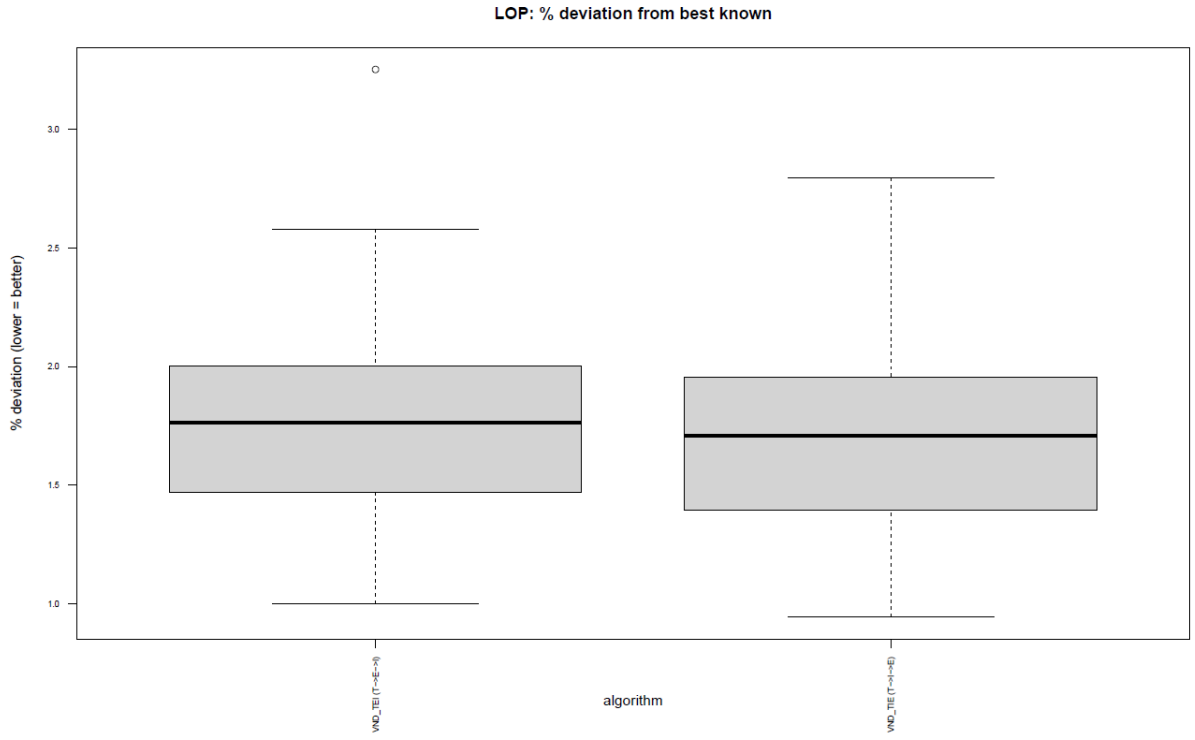
The main source code is organized within a Visual Studio solution (.slnx).
Since the project is named LOP, all source files are located in **./LOP/LOP/**. On **macOS**, this may instead be **./LOP-main/LOP/**.
This directory contains:

- All **C source files** (.c)
- All **C header files** (.h)
- All **R scripts** used for statistical testing

README:

The README.md file is located in the root of the project directory: **./LOP/**




LOP: % deviation from best known
% deviation (lower = better)
3.0
2.5
2.0
1.5
1.0
VND_TEI (T->E->I)
VND_TIE (T->I->E)
algorithm